

COLLEGE

UNIT 3: PLOTTING USING PYLAB

511-1474

Plotting Using PyLab

```
python -m pip install matplotlib
```

PyLab is a procedural interface to the Matplotlib object-oriented plotting library. Matplotlib is the whole package; matplotlib.pyplot is a module in Matplotlib; and PyLab is a module that gets installed alongside Matplotlib.

PyLab is a convenience module that bulk imports matplotlib.pyplot (for plotting) and NumPy (for Mathematics and working with arrays) in a single name space.

PyLab is a Python standard library module that provides many of the facilities of MATLAB. It is “a high- level technical computing language and interactive environment for algorithm development, data visualization, data analysis, and numeric computation.”

Line Plot

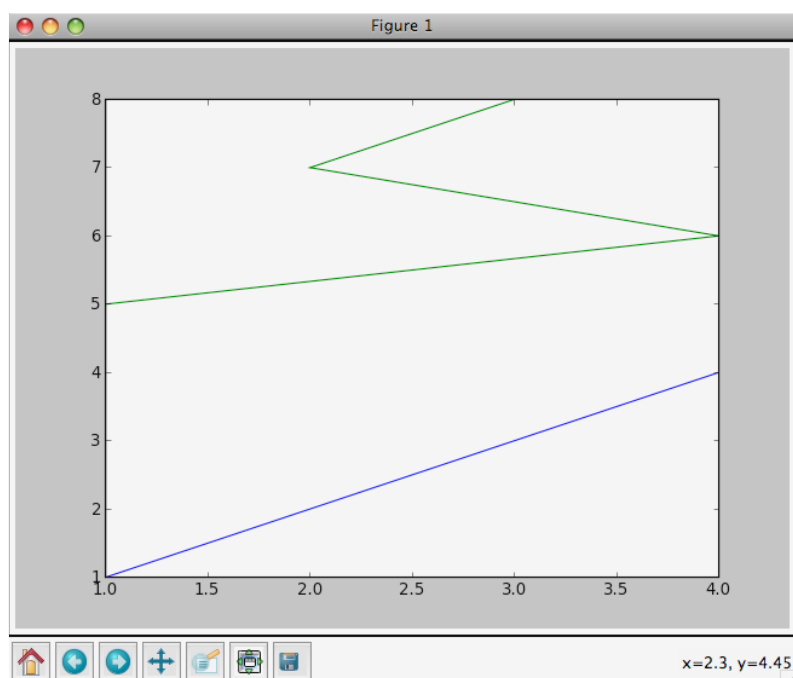
A common type of graph to plot is a line relating x-values to particular y-values.

A simple example

Let's start with a simple example that uses pylab.plot to produce two plots.

```
import pylab
pylab.plot([1,2,3,4], [1,2,3,4])
pylab.plot([1,4,2,3], [5,6,7,8])
pylab.show()
```

Executing the code will cause a window to appear on your computer monitor. Its exact appearance will depend on the operating system on your machine, but it will always look something like:



Sequences of the same length

The bar at the top contains the name of the window, in this case “Figure 1.” In this example, the name is generated automatically by pylab.

The middle section of the window contains the plot generated by the two lines of code following the import statement. The bottom line was generated by the statement `pylab.plot([1,2,3,4], [1,2,3,4])`. Notice that the two arguments are lists of the same length. Together, they provide a sequence of four $\langle x, y \rangle$ coordinate pairs, $[(1,1), (2,2), (3,3), (4,4)]$. These are plotted in order, and then connected by a line. (Pylab automatically chooses the line color for each plot command, but this can be overridden by an optional argument, as we shall see.) The zig zag plot at the top of the middle section is produced the same way. It zig zags

because the sequence of Cartesian points being connected is $[(1,5), (4,6), (2,7), (3,8)]$.

`pylab.show()`

The final line of code, `pylab.show()` causes the window to appear on the screen. If that line were not there, the figure would still have been produced, but it would not have been displayed.

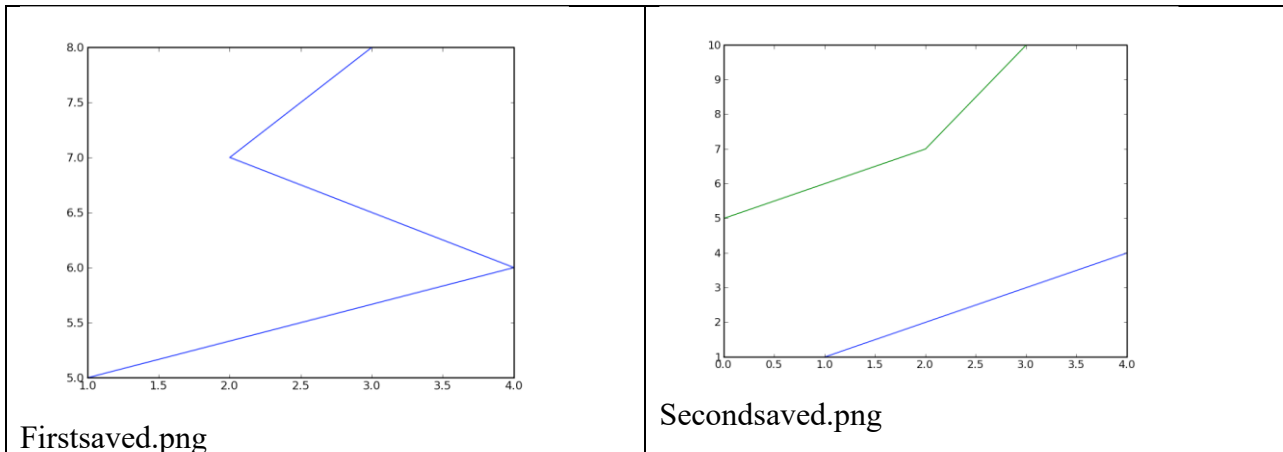
Introduction of the Buttons:

The bar at the bottom of the window contains a number of push buttons. The rightmost button is used to write the plot to a file.³ The next button over is used to adjust the appearance of the plot in the window. This is useful when there is more than one plot per figure. The next four buttons are used for panning and zooming. And the button on the left is used to restore the figure to its original appearance after you are done playing with pan and zoom.

The multiple figures

It is possible to produce more than one figure and to write them to files rather than the display. The following code produces and saves to files named `firstSaved.png` and `secondSaved.png` the two plots.

```
import pylab
pylab.figure(1)
pylab.plot([1,2,3,4], [1,2,3,4])
pylab.figure(2)
pylab.plot([1,4,2,3], [5,6,7,8])
pylab.savefig('firstSaved')
pylab.figure(1)
pylab.plot([5,6,7,10])
pylab.savefig('secondSaved')
```



One argument

Observe that the last call to `pylab.plot` is passed only one argument. This argument supplies the Y values. The corresponding X values are `range(len(Y-values))`, which is why they range from 0 to 3 in this case.

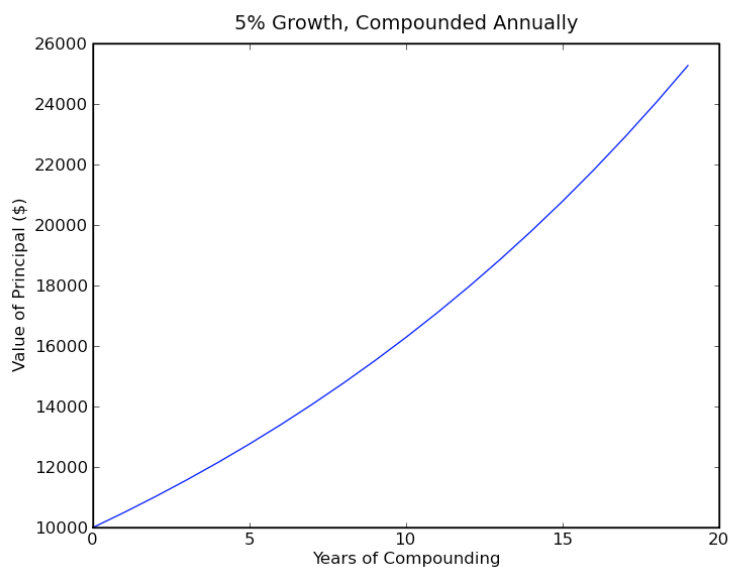
“Current figure.”

PyLab has a notion of “current figure.” Executing the statement `pylab.figure(x)` sets the current figure to the figure numbered `x`. Subsequently executed `pylab` commands implicitly refer to that figure, until another `pylab.figure` command is executed. This explains why the plot on the left, which corresponds to the figured numbered 2, is stored in `firstSaved.png`.

Compound interest Example with Informative Titles:

```
import pylab
principal = 10000 #initial investment
interestRate = 0.05
years = 20
values = []
for i in range(years + 1):
    values.append(principal)
    principal += principal*interestRate
pylab.title('5% Growth, Compounded Annually')
pylab.xlabel('Years of Compounding')
pylab.ylabel('Value of Principal ($)')
pylab.plot(values)
pylab.show()
```

The code produces the plot:

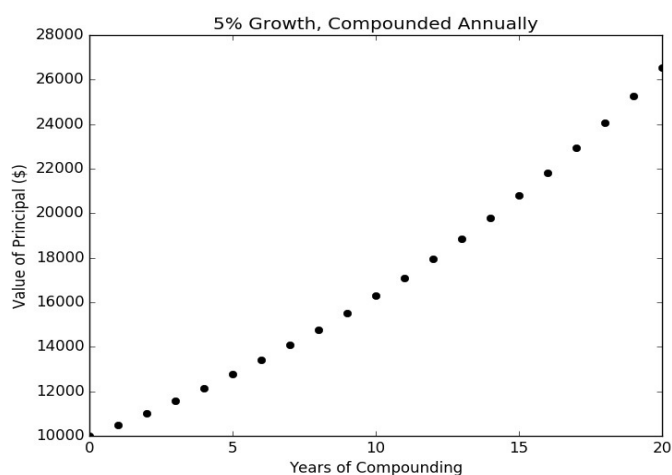


Optional argument

For every plotted curve, there is an optional argument that is a format string indicating the color and line type of the plot. The letters and symbols of the format string are derived from those used in MATLAB, and are composed of a color indicator followed by an optional line-style indicator. The default format string is 'b-', which produces a solid blue line. To plot the the growth in principal with black circles, one would replace the call

```
pylab.plot(values, 'ko', )
```

which produces the plot:



The following format string characters are accepted to control the line style or marker:

character	description
'-'	solid line style
'--'	dashed line style
'-.'	dash-dot line style

character	description
'.'	dotted line style
'.'	point marker
','	pixel marker
'o'	circle marker
'v'	triangle_down marker
'^'	triangle_up marker
'<'	triangle_left marker
'>'	triangle_right marker
'1'	tri_down marker
'2'	tri_up marker
'3'	tri_left marker
'4'	tri_right marker
's'	square marker
'p'	pentagon marker
'*'	star marker
'h'	hexagon1 marker
'H'	hexagon2 marker
'+'	plus marker
'x'	x marker
'D'	diamond marker
'd'	thin_diamond marker
' '	vline marker
'_'	hline marker

Color abbreviations:

character	color
'b'	blue
'g'	green
'r'	red
'c'	cyan
'm'	magenta

character	color
'y'	yellow
'k'	black
'w'	white

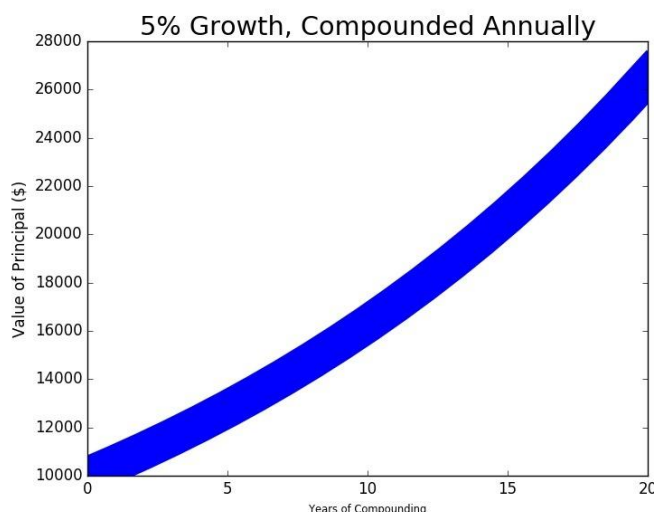
Change the type size and line width

It is also possible to change the type size and line width used in plots. This can be done using keyword arguments in individual calls to functions.

E.g., the code:

```
principal = 10000 #initial
investment interestRate = 0.05
years = 20
values = []
for i in range(years + 1):
    values.append(principal)
    principal += principal*interestRate
pylab.plot(values, linewidth = 30)
pylab.title('5% Growth, Compounded Annually', fontsize = 'xx-large')
pylab.xlabel('Years of Compounding', fontsize = 'x-small')
pylab.ylabel('Value of Principal ($)')
```

produces the following plot:



Bar Plot

A **bar plot** (or bar chart) is a graphical representation that uses rectangular bars to compare different categories. The height or length of each bar corresponds to the value it represents. The x-axis typically shows the categories being compared, while the y-axis shows the values

associated with those categories. This visual format makes it easy to compare quantities across different groups.

This function takes several parameters:

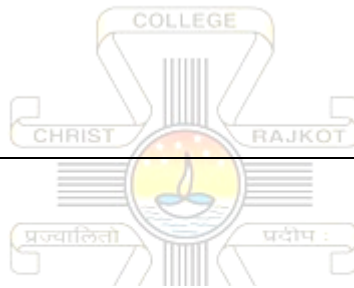
- **x:** The categories (e.g., fruits).
- **height:** The corresponding values (e.g., sales).
- **width:** The width of the bars (default is 0.8).
- **bottom:** The baseline for the bars (default is 0).
- **align:** How to align bars ('center' or 'edge')

Example

```
import matplotlib.pyplot as plt
import numpy as np

fruits = ['Apples', 'Bananas', 'Cherries', 'Dates']
sales = [400, 350, 300, 450]

plt.bar(fruits, sales, color='violet') #color is optional
plt.title('Fruit Sales')
plt.xlabel('Fruits')
plt.ylabel('Sales')
plt.show()
```



Output:



Scatter plots

Scatter plots are one of the most fundamental and powerful tools for visualizing relationships between two numerical variables. `matplotlib.pyplot.scatter()` plots points on a Cartesian plane defined by X and Y coordinates.

Syntax

```
matplotlib.pyplot.scatter(x, y, s=None, c=None, marker=None, cmap=None, alpha=None,
edgecolors=None, label=None)
```

Parameters:

Parameter	Description
x, y	Sequences of data points to plot
s	Marker size (scalar or array-like)
c	Marker color
marker	Shape of the marker
cmap	Colormap for mapping numeric values to colors
alpha	Transparency (0 = transparent, 1 = opaque)
edgecolors	Color of marker edges
label	Legend label for the dataset

Example:

```
x1 = np.array([160, 165, 170, 175, 180, 185, 190, 195, 200, 205])
y1 = np.array([55, 58, 60, 62, 64, 66, 68, 70, 72, 74])

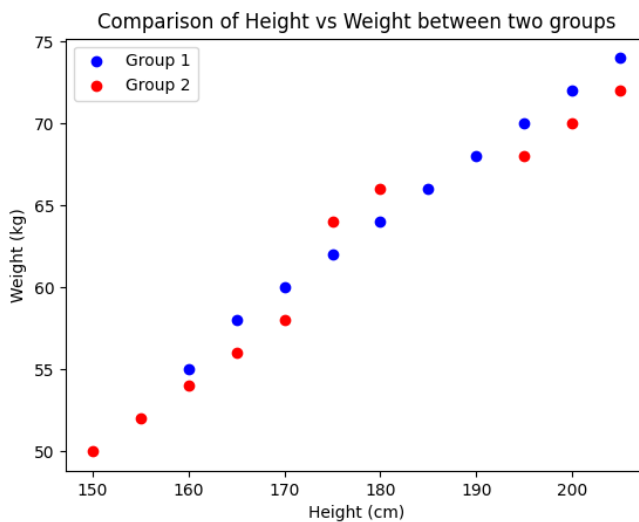
x2 = np.array([150, 155, 160, 165, 170, 175, 180, 195, 200, 205])
y2 = np.array([50, 52, 54, 56, 58, 64, 66, 68, 70, 72])

plt.scatter(x1, y1, color='blue', label='Group 1')
plt.scatter(x2, y2, color='red', label='Group 2')

plt.xlabel('Height (cm)')
```

```
plt.ylabel('Weight (kg)')
plt.title('Comparison of Height vs Weight between two groups')
plt.legend()
plt.show()
```

Output:



Histograms

Histograms are one of the most fundamental tools in data visualization. They provide a graphical representation of data distribution, showing how frequently each value or range of values occurs. Histograms are especially useful for analyzing continuous numerical data, such as measurements, sensor readings, or experimental results.

A histogram is a type of bar plot where:

- The X-axis represents intervals (called bins) of the data.
- The Y-axis represents the frequency of values within each bin.

Unlike regular bar plots, histograms group data into bins to summarize data distribution effectively.

Creating a Matplotlib Histogram

1. Divide the data range into consecutive, non-overlapping intervals called bins.
2. Count how many values fall into each bin.
3. Use the `matplotlib.pyplot.hist()` function to plot the histogram.

The following table shows the parameters accepted by `matplotlib.pyplot.hist()` function :

Attribute	Parameter
x	Array or sequence of numerical data.
bins	Number of bins (int) or specific intervals (array).

Attribute	Parameter
density	If True, normalizes histogram to show probability instead of frequency.
range	Tuple specifying lower and upper limits of bins.
histtype	Type of histogram: bar, barstacked, step, stepfilled. Default: bar.
align	Bin alignment: left, right, mid.
weights	Array of weights for each data point.
bottom	Baseline for bins.
rwidth	Relative width of bars (0–1).
color	Color of bars. Can be a single color or sequence.
label	Label for legend.
log	If True, uses logarithmic scale on Y-axis

Example

```
import matplotlib.pyplot as plt
import numpy as np

# Generate random data for the histogram
data = np.random.randn(1000)

# Plotting a basic histogram
plt.hist(data, bins=30, color='skyblue', edgecolor='black')

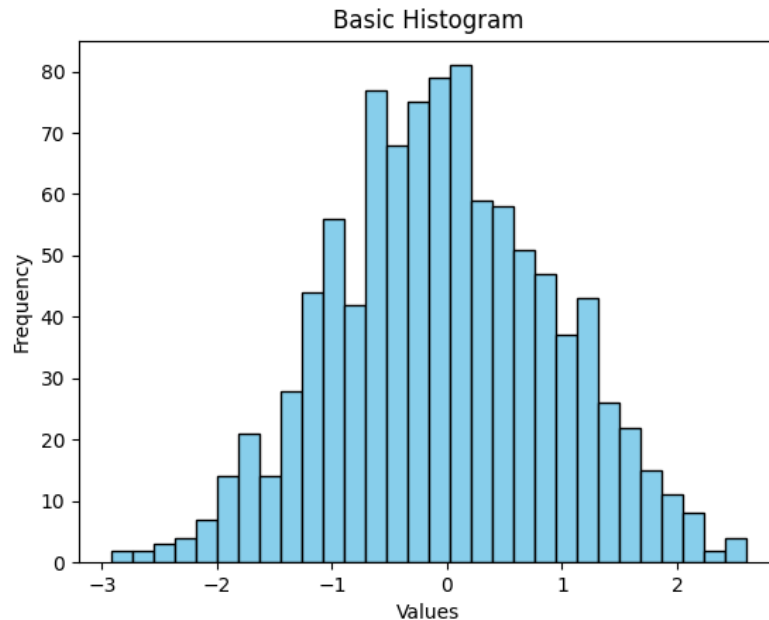
# Adding labels and title
plt.xlabel('Values')
plt.ylabel('Frequency')
```

```
plt.title('Basic Histogram')
```

```
# Display the plot
```

```
plt.show()
```

It will produce following output:



Explanation:

- Generates 1000 random numbers from a standard normal distribution.
- Plots a histogram with 30 bins, sky-blue bars, and black edges.
- Adds X and Y axis labels and a title.
- Displays the histogram plot.

Pie Chart

A Pie Chart is a circular statistical plot that can display only one series of data. The area of the chart is the total percentage of the given data. Pie charts in Python are widely used in business presentations, reports, and dashboards due to their simplicity and effectiveness in displaying data distributions.

Basic Structure of a Pie Chart

A pie chart consists of slices that represent different categories. The size of each slice is proportional to the quantity it represents. The following components are essential when creating a **pie chart in Matplotlib**:

- **Data:** The values or counts for each category.
- **Labels:** The names of each category, which will be displayed alongside the slices.
- **Colors:** Optional, but colors can be used to differentiate between slices effectively.

Matplotlib API has pie() function in its pyplot module which create a pie chart representing the data in an array. let's create pie chart in python.

Syntax: matplotlib.pyplot.pie(data, explode=None, labels=None, colors=None, autopct=None, shadow=False)

Parameters:

- **data** represents the array of data values to be plotted, the fractional area of each slice is represented by **data/sum(data)**
- **labels** is a list of sequence of strings which sets the label of each wedge.
- **color** attribute is used to provide color to the wedges.
- **autopct** is a string used to label the wedge with their numerical value.
- **shadow** is used to create shadow of wedge.

Example

```
# Import libraries
from matplotlib import pyplot as plt
import numpy as np

# Creating dataset
cars = ['AUDI', 'BMW', 'FORD',
        'TESLA', 'JAGUAR', 'MERCEDES']

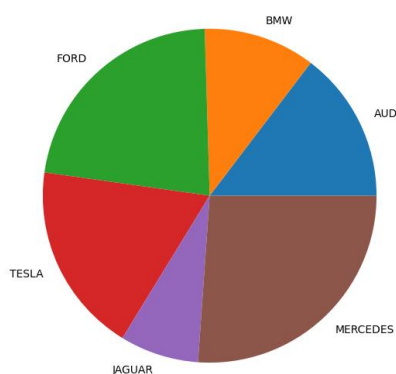
data = [23, 17, 35, 29, 12, 41]

# Creating plot
fig = plt.figure(figsize=(10, 7))
plt.pie(data, labels=cars)

# show plot
plt.show()
```



Output:



Summary of all Graphs

List of different methods:

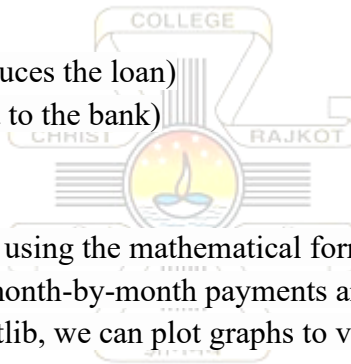
1. `pylab.plot()`
2. `pylab.scatter()`
3. `pylab.bar()`
4. `pylab.hist()`
5. `pylab.pie()`

Common methods for all the types of graphs:

1. `pylab.title()`
2. `pylab.xlabel()`
3. `pylab.ylabel()`
4. `pylab.show()`

Mortgages

- A mortgage is a loan taken from a bank (usually to buy a house).
- The borrower pays back the loan every month in fixed installments, called EMI (Equated Monthly Installments).
- Each EMI has two parts:
 - **Principal** (amount that reduces the loan)
 - **Interest** (extra money paid to the bank)



Use Python for Mortgage

- Python can calculate EMIs using the mathematical formula.
- Python can also simulate month-by-month payments and show how the loan decreases.
- Using libraries like matplotlib, we can plot graphs to visually see how balance reduces with time

Formula for EMI

$$EMI = \frac{P \times r}{1 - (1 + r)^{-n}}$$

Where:

- P = Loan Amount (Principal)
- r = Monthly Interest Rate (Annual Rate / 12)
- n = Number of Months

Steps in Python Program

1. Take loan amount, interest rate, and years.
2. Convert years into months.

3. Use EMI formula to calculate fixed monthly payment.
4. Create a loop for each month:
 - Calculate interest for that month.
 - Subtract principal part from balance.
 - Store remaining balance.
5. Plot the balances using matplotlib.

To install mortgage:

```
python -m pip install mortgage
```

Example:

```
from mortgage import Loan
```

```
loan = Loan(principal=200000, interest=.06, term=30)
loan.summarize
```

output:

Original Balance: \$ 200,000

Interest Rate: 0.06 %

APY: 6.17 %

APR: 6.00 %

Term: 30 years

Monthly Payment: \$ 1199.10

Total principal payments: \$ 200,000.00

Total interest payments: \$ 231,676.38

Total payments: \$ 431,676.38

Interest to principal: 115.8 %

Years to pay: 30.0

Plotting Mortgages with Extended Examples

Example 1

```
import matplotlib.pyplot as plt
# Loan details
loan_amount = 100000 # total loan (₹1,00,000)
annual_interest_rate = 0.08 # 8% yearly interest
years = 10 # loan term (10 years)

# Calculations
months = years * 12
monthly_interest_rate = annual_interest_rate / 12

# EMI formula
```

```
EMI = loan_amount * monthly_interest_rate / (1 - (1 + monthly_interest_rate) ** -months)
```

```
balance = loan_amount
```

```
balances = [] # to store balance after each month
```

```
for m in range(months):
```

```
    interest = balance * monthly_interest_rate
```

```
    principal = EMI - interest
```

```
    balance -= principal
```

```
    balances.append(balance)
```

```
# Print details
```

```
print("Loan Amount: ₹", loan_amount)
```

```
print("Loan Term:", years, "years (", months, "months)")
```

```
print("Annual Interest Rate:", annual_interest_rate * 100, "%")
```

```
print("Monthly EMI: ₹", round(EMI, 2))
```

```
print("Total Amount Paid: ₹", round(EMI * months, 2))
```

```
print("Total Interest Paid: ₹", round(EMI * months - loan_amount, 2))
```

```
# Plotting
```

```
plt.plot(balances)
```

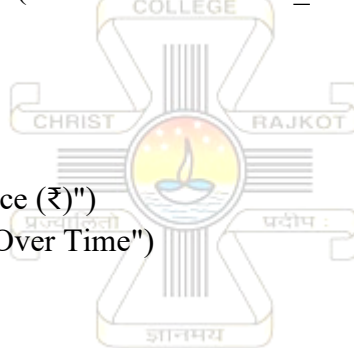
```
plt.xlabel("Months")
```

```
plt.ylabel("Remaining Loan Balance (₹)")
```

```
plt.title("Mortgage Loan Balance Over Time")
```

```
plt.grid(True)
```

```
plt.show()
```



Output:

Loan Amount: ₹ 100000

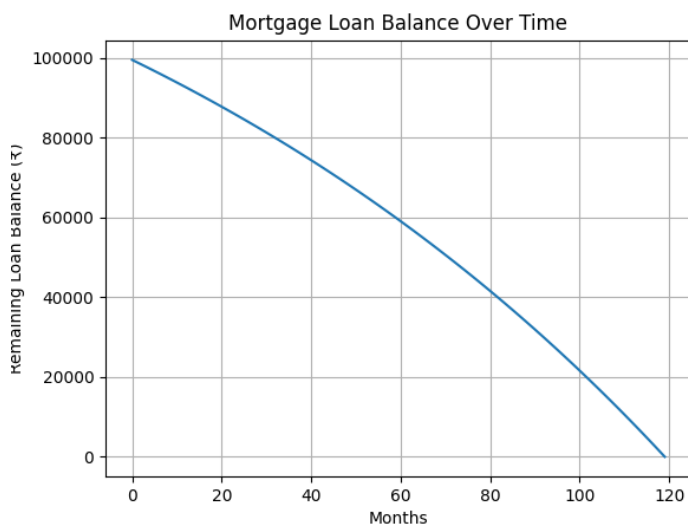
Loan Term: 10 years (120 months)

Annual Interest Rate: 8.0 %

Monthly EMI: ₹ 1213.28

Total Amount Paid: ₹ 145593.11

Total Interest Paid: ₹ 45593.11



Example 2

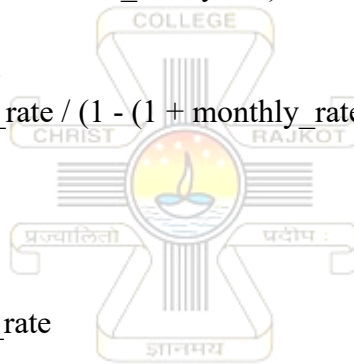
```
import matplotlib.pyplot as plt
def mortgage_balance(loan_amount, annual_rate, years):
    months = years * 12
    monthly_rate = annual_rate / 12
    EMI = loan_amount * monthly_rate / (1 - (1 + monthly_rate) ** -months)

    balance = loan_amount
    balances = []
    for m in range(months):
        interest = balance * monthly_rate
        principal = EMI - interest
        balance -= principal
        balances.append(balance)

    # Print details
    print("Loan Amount: ₹", loan_amount)
    print("Loan Term:", years, "years (", months, "months)")
    print("Annual Interest Rate:", annual_rate * 100, "%")
    print("Monthly EMI: ₹", round(EMI, 2))
    print("Total Amount Paid: ₹", round(EMI * months, 2))
    print("Total Interest Paid: ₹", round(EMI * months - loan_amount, 2))
    print("-" * 40)
    return balances

# Loan details
loan = 100000 # ₹1,00,000

# Mortgage 1: 10 years, 8% interest
```



```

balances1 = mortgage_balance(loan, 0.08, 10)
# Mortgage 2: 15 years, 6% interest
balances2 = mortgage_balance(loan, 0.06, 15)
# Plot both
plt.plot(balances1, label="10 yrs @ 8%")
plt.plot(balances2, label="15 yrs @ 6%")

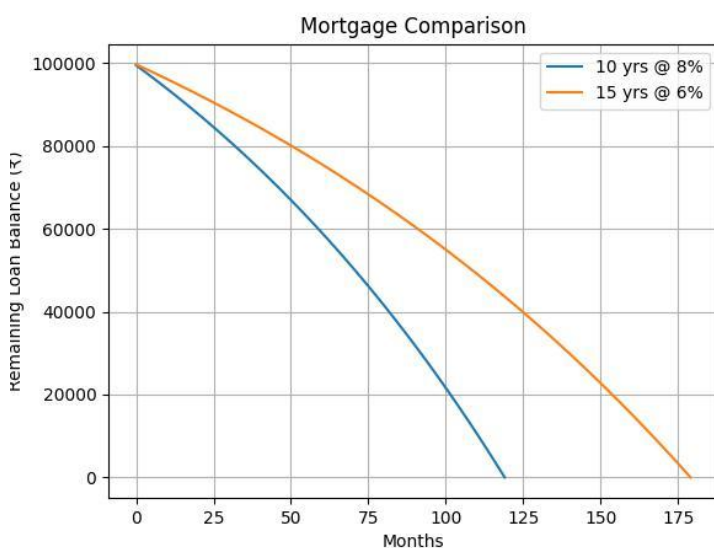
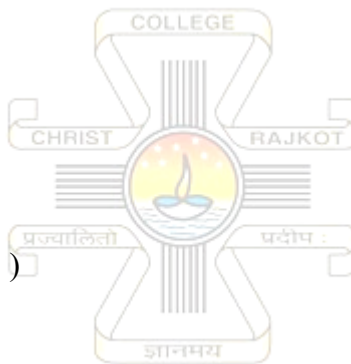
plt.xlabel("Months")
plt.ylabel("Remaining Loan Balance (₹)")
plt.title("Mortgage Comparison")
plt.legend()
plt.grid(True)
plt.show()

```

Output:

Loan Amount: ₹ 100000
 Loan Term: 10 years (120 months)
 Annual Interest Rate: 8.0 %
 Monthly EMI: ₹ 1213.28
 Total Amount Paid: ₹ 145593.11
 Total Interest Paid: ₹ 45593.11

 Loan Amount: ₹ 100000
 Loan Term: 15 years (180 months)
 Annual Interest Rate: 6.0 %
 Monthly EMI: ₹ 843.86
 Total Amount Paid: ₹ 151894.23
 Total Interest Paid: ₹ 51894.23



Fibonacci Sequences, Revisited

A straightforward recursive implementation of the Fibonacci function is shown here :

```
def fib(n):
    if n == 0 or n == 1:
        return 1
    else:
        return fib(n-1) + fib(n-2)
n=10
print("series")
for i in range(n):
    print(fib(i))
```

While this implementation of the recurrence is obviously correct, it is terribly inefficient. Try, for example, running `fib(120)`, but don't wait for it to complete. The complexity of the implementation is a bit hard to derive. That is, its growth is proportional to the growth in the value of the result, and the growth rate of the Fibonacci sequence is substantial. For example, total of `fib(120)` is very large number. If each recursive call took a nanosecond, `fib(120)` would take a long time to execute. Let's try and figure out why this implementation takes so long. Given the tiny amount of code in the body of `fib`, it's clear that the problem must be the number of times that `fib` calls itself. As an example, look at the tree of calls associated with the invocation `fib(6)`.

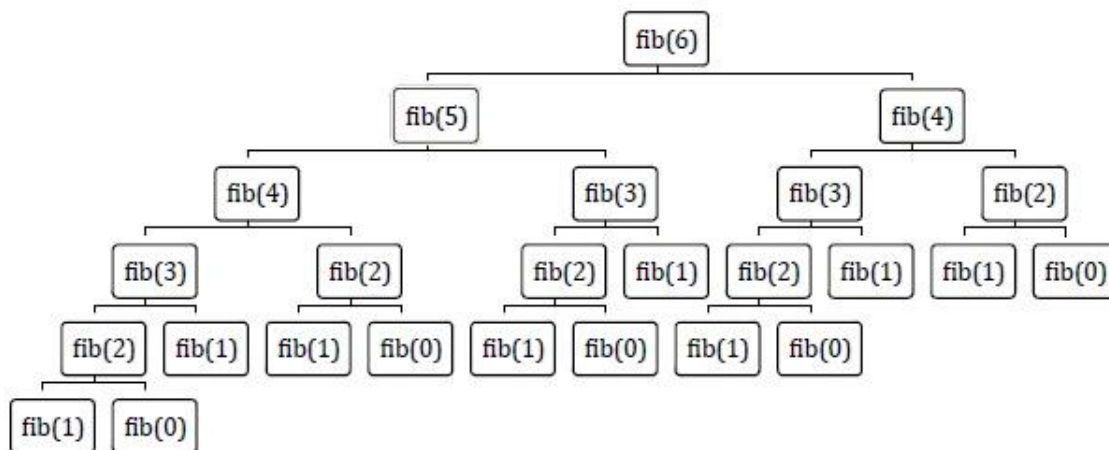


Figure 18.2 Tree of calls for recursive Fibonacci

Notice that we are computing the same values over and over again. For example `fib` gets called with 3 three times, and each of these calls provokes four additional calls of `fib`. It doesn't require a genius to think that it might be a good idea to record the value returned by the first call, and then look it up rather than compute it each time it is needed. This is called **memoization**, and is the key idea behind dynamic programming.

Figure below contains an implementation of Fibonacci based on this idea. The function `fastFib` has a parameter, `memo`, that it uses to keep track of the numbers it has already evaluated. The parameter has a default value, the empty dictionary, so that clients of `fastFib` don't have to worry about supplying an initial value for `memo`. When `fastFib` is called with an `n > 1`, it attempts to

look up n in memo. If it is not there (because this is the first time fastFib has been called with that value), an exception is raised. When this happens, fastFib uses the normal Fibonacci recurrence, and then stores the result in memo.

```
def fastFib(n, memo = {}):
    """Assumes n is an int >= 0, memo used only by recursive calls Returns Fibonacci of n"""
    if n == 0 or n == 1:
        return 1
    try:
        return memo[n]
    except KeyError:
        result = fastFib(n-1, memo) + fastFib(n-2, memo)
        memo[n] = result
        return result

n=20
print("series")
for i in range(n):
    print(fastFib(i))
```

If you try running fastFib, you will see that it is indeed quite fast, that is, fastfib(120) returns almost instantly. It calls fib exactly once for each value from 0 to n .

Dynamic programming and The 0/1 Knapsack algorithm

What is Dynamic Programming?

- Dynamic Programming (DP) is a technique to solve problems by breaking them into smaller subproblems, solving each once, and reusing the results (memoization or tabulation).
- It avoids repeated calculations and makes algorithms much faster.
- DP is useful in optimization problems (finding max, min, shortest path, etc.).

The 0/1 Knapsack algorithm

Knapsack is a classic problem that involves decision-making based on constraints. Suppose we have a knapsack with fixed capacity, and items with fixed values and sizes (it might be volume of the item or weight).

We need to add items into the knapsack such that we pack the maximum value items. From the available combinations, we need to provide an optimized solution. There are three types of knapsack problems:

- **0-1 Knapsack:** We can either pick an item or not (0 or 1).
- **Fractional Knapsack:** We can pick a part of an item.
- **Unbounded Knapsack:** We can choose items multiple times.

We can solve the 0-1 Knapsack problem using a simple recursive function, but this takes more time as the number of items increases. Using the dynamic programming technique, we can solve this problem faster.

Example:

```
val = [50, 100, 150, 200]
wt = [8, 16, 32, 40]
W = 64
```

Combinations:

Weight	Values	Less than or equal to W(64)
8+16+32=56	50+100+150=300	56<=64
8+16+40=64	50+100+200=350	64<=64

The second combination is considered to be perfect as the W is satisfied and maximum value is taken.

0-1 Knapsack Problem Statement

You are given weights (**w[i]**) and values (**v[i]**) of **n** items. You need to fill a bag of capacity **W** with these items such that the total value is maximized. However, the weight of the items placed inside should not exceed the bag's capacity. Each item can be chosen only once.

Recursive (Brute-force) Solution

Here we will solve the 0-1 Knapsack problem using a recursive function that explores all possible combinations of including or excluding each item. This checks every choice to find the one that gives the maximum value.

Steps for the Recursive Approach:

- If no items are left or capacity becomes 0, return 0.
- If the weight of the current item is more than the capacity of the Knapsack, skip the item.
- Otherwise, calculate the maximum value by including the current item or excluding it.
- Return the maximum of these two values.

Example

In the recursive function, we return the maximum value possible and solve sub-problems such as "no items left or, capacity is exhausted", "weight of an item is more than the capacity of the bag".

```
# Returns the maximum value that can be stored in the bag
def knapSack(W, wt, val, n):
    # Base condition: no items left or no remaining capacity
    if n == 0 or W == 0:
        return 0
    # If weight of nth item is more than capacity, skip it
    if wt[n-1] > W:
        return knapSack(W, wt, val, n-1)
    # Return the maximum of two cases, nth item included or, excluded
    else:
        return max(
```

```

        val[n-1] + knapSack(W - wt[n-1], wt, val, n-1),
        knapSack(W, wt, val, n-1)
    )
# Test input
val = [50, 100, 150, 200]
wt = [8, 16, 32, 40]
W = 64
n = len(val)
print(knapSack(W, wt, val, n))

```

Output:

350

Dynamic Programming (Bottom-Up)

The recursive approach solves the same sub-problems multiple times. Dynamic Programming avoids this by storing the results of sub-problems in a 2D table and building the solution step by step.

Steps for the Dynamic Programming Approach:

- Create a table **K[n+1][W+1]** initialized to 0.
- Iterate through each item and each capacity from 0 to W.
- If the current item's weight is less than or equal to the current capacity, set **K[i][w]** as the max of including or excluding the item.
- Else, carry forward the previous value without including the item.
- After filling the table, the value at **K[n][W]** is the result.

Example

In this example, the table **K[i][w]** represents the maximum value achievable using the first **i** items with a total weight limit **w**. We iteratively fill this table to find the solution -

A Dynamic Programming based Python Program for 0-1 Knapsack problem

```

def knapSack(W, wt, val, n):
    # Create a 2D table to store results of subproblems
    K = [[0 for _ in range(W + 1)] for _ in range(n + 1)]

    # Build the table K[][] in bottom-up manner
    for i in range(n + 1):
        for w in range(W + 1):
            if i == 0 or w == 0:
                K[i][w] = 0
            elif wt[i-1] <= w:
                K[i][w] = max(val[i-1] + K[i-1][w - wt[i-1]], K[i-1][w])
            else:
                K[i][w] = K[i-1][w]

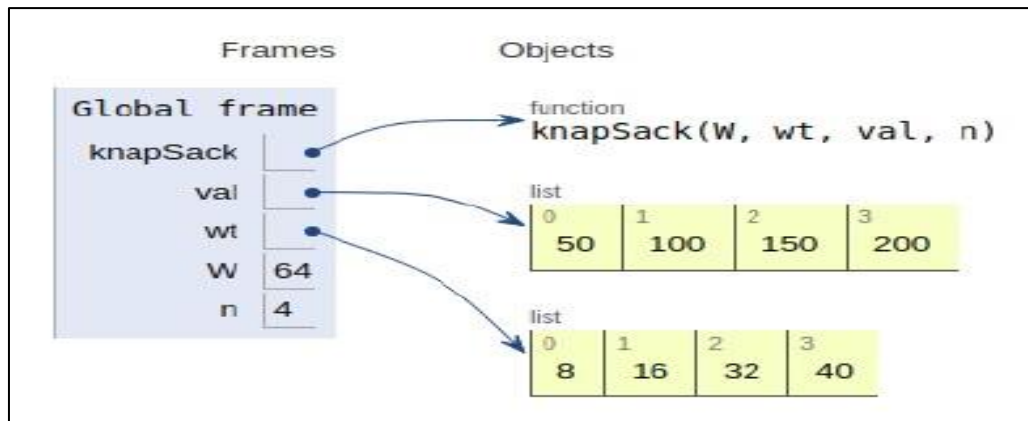
    return K[n][W]

```

```
# Test input
val = [50, 100, 150, 200]
wt = [8, 16, 32, 40]
W = 64
n = len(val)
print(knapSack(W, wt, val, n))
```

Output:

350



Divide and conquer

In divide and conquer approach, the problem in hand, is divided into smaller sub-problems and then each problem is solved independently. When we keep on dividing the subproblems into even smaller sub-problems, we may eventually reach a stage where no more division is possible. Those "atomic" smallest possible sub-problem (fractions) are solved. The solution of all sub-problems is finally merged in order to obtain the solution of an original problem.

Broadly, we can understand **divide-and-conquer** approach in a three-step process.

Divide/Break

This step involves breaking the problem into smaller sub-problems. Sub-problems should represent a part of the original problem. This step generally takes a recursive approach to divide the problem until no sub-problem is further divisible. At this stage, sub-problems become atomic in nature but still represent some part of the actual problem.

Conquer/Solve

This step receives a lot of smaller sub-problems to be solved. Generally, at this level, the problems are considered 'solved' on their own.

Merge/Combine

When the smaller sub-problems are solved, this stage recursively combines them until they formulate a solution of the original problem. This algorithmic approach works recursively and conquer & merge steps work so close that they appear as one.



Differentiate between Divide & Conquer Method vs Dynamic Programming.

Divide & Conquer Method	Dynamic Programming
1.It deals (involves) three steps at each level of recursion: Divide the problem into a number of subproblems. Conquer the subproblems by solving them recursively. Combine the solution to the subproblems into the solution for original subproblems.	1.It involves the sequence of four steps: ○ Characterize the structure of optimal solutions. ○ Recursively defines the values of optimal solutions. ○ Compute the value of optimal solutions in a Bottom-up minimum. ○ Construct an Optimal Solution from computed information.
2. It is Recursive.	2. It is non Recursive.
3. It does more work on subproblems and hence has more time consumption.	3. It solves subproblems only once and then stores in the table.
4. It is a top-down approach.	4. It is a Bottom-up approach.
5. In this subproblems are independent of each other.	5. In this subproblems are interdependent.
6. For example: Merge Sort & Binary Search etc.	6. For example: Matrix Multiplication.